🧭 Django Production Engineer Roadmap

1️⃣ Django Fundamentals (Core)

প্রথমে Django খুব solid করতে হবে।

শিখতে হবে:

- Project structure
- Apps
- Models
- Views
- Templates
- URL routing
- Forms
- Admin panel
- Authentication system

Example project:

```
Blog System
User Authentication
```

Goal:

```
Django project fully understand
```

2️⃣ Django ORM + Database Design

এটা backend engineer এর জন্য খুব important।

শিখতে হবে:

- Model relationships
  - OneToOne
  - ForeignKey
  - ManyToMany
- Query optimization

```
select_related
prefetch_related
```

- Database indexing
- Migrations

Example project:

```
E-commerce database
Users
Orders
Products
```

3️⃣ Django REST Framework (DRF)

Modern backend API development।

শিখতে হবে:

- Serializer
- APIView
- ViewSets
- Routers
- Pagination
- Filtering
- Authentication
- Permissions

Example API:

```
User API
Product API
Order API
```

Goal:

```
Full REST API backend
```

4️⃣ PostgreSQL

Production Django apps SQLite use করে না।

শিখতে হবে:

- PostgreSQL install
- Django connection
- JSON fields
- indexing
- query optimization

Example:

```
Django + PostgreSQL project
```

5️⃣ Redis

Redis use হয়:

- Cache
- Task queue
- Rate limiting
- Sessions

শিখতে হবে:

```
django-redis
caching system
```

Example:

```
API response caching
```

6️⃣ Celery (Background Tasks)

Production systems এ heavy tasks background এ run হয়।

শিখতে হবে:

- Celery setup
- Redis broker
- Task queue

- periodic tasks

Example tasks:

```
send email
generate report
image processing
```

7️⃣ Docker

Deployment এর জন্য essential।

শিখতে হবে:

- Dockerfile
- Docker compose
- Multi container apps

Example stack:

```
Django
Postgres
Redis
Celery
```

8️⃣ Nginx + Gunicorn

Production server setup।

Flow:

```
User
  |
  ▼
Nginx
  |
  ▼
Gunicorn
  |
  ▼
Django
```

শিখতে হবে:

- Reverse proxy
- Static file serving
- SSL setup

9️⃣ Production Deployment

Deploy করতে শিখতে হবে।

Popular platforms:

- AWS
- DigitalOcean
- Railway
- Render
- VPS

Deployment stack:

```
Docker
Nginx
Gunicorn
PostgreSQL
Redis
Celery
```

🔟 Advanced Production Skills

এগুলো backend engineer level skills।

শিখতে হবে:

- API rate limiting
- logging
- monitoring
- CI/CD
- scaling
- security

💻 Final Production Architecture

```
          User
           |
           ▼
         Nginx
           |
           ▼
        Gunicorn
           |
           ▼
         Django
        /      \
       ▼        ▼
  PostgreSQL   Redis
                 |
                 ▼
               Celery
```

🎯 Final Goal

তুমি এই level project build করতে পারবে:

```
Fullstack SaaS app
AI system backend
Real-time API
Microservice backend
```

ভালো 👍 তাহলে আমি তোমাকে **Real Production Project Plan** দিচ্ছি যেটা করলে তুমি সত্যিই **Django Production Engineer level skill** পাবে।
তোমার আগের **AI CCTV / Suspicious Detection project** এর সাথে এটা perfectly match করবে।

🚀 Project: AI Surveillance System (Production Level)

এই project এ থাকবে:

```
AI Model
   |
   ▼
Django API
```

```
    |
    ▼
PostgreSQL
    |
    ▼
React Dashboard
    |
    ▼
Nginx
```

Background system:

```
Django
  |
  ▼
Redis
  |
  ▼
Celery Worker
```

🧱 Step 1 — Django Backend

Backend structure:

```
backend
 ├── config
 ├── cameras
 ├── alerts
 ├── users
 └── ai_detection
```

Features:

- camera API
- alert API
- suspicious detection API
- authentication

Example models:

```
class Camera(models.Model):
    name = models.CharField(max_length=100)
    location = models.CharField(max_length=200)
```

🧠 Step 2 — AI Model Integration

তোমার **CNN + LSTM suspicious detection model** এখানে integrate হবে।

Flow:

```
Camera frame
     |
     ▼
AI model
     |
     ▼
Prediction
     |
     ▼
Django API
```

Example API:

```
POST /api/detect
```

🗄 Step 3 — PostgreSQL Database

Database tables:

```
Users
Cameras
Alerts
Detection Logs
```

Example:

```
Alert
-----
id
camera_id
timestamp
confidence
```

⚡ Step 4 — Redis + Celery

Background tasks:

```
Video processing
Alert sending
Email notifications
Report generation
```

Example:

```
@shared_task
def process_video(video_id):
    run_ai_model()
```

📊 Step 5 — React Dashboard

Dashboard features:

```
Live camera list
Alert notifications
Detection history
Statistics
```

Example UI pages:

```
Dashboard
Camera page
Alerts page
Analytics page
```

🐳 Step 6 — Docker

Containers:

```
backend
frontend
postgres
redis
celery
nginx
```

Example docker compose:

```
services
 - backend
 - frontend
 - postgres
 - redis
 - celery
 - nginx
```

🌐 Step 7 — Nginx

Nginx handle করবে:

```
React static files
API proxy
SSL
Load balancing
```

Flow:

```
User
 │
 ▼
Nginx
 │
 ├── React
 └── Django API
```

📈 Step 8 — Production Deployment

Deploy করতে পারো:

```
AWS EC2
DigitalOcean
VPS
```

Deployment stack:

```
Docker
Nginx
Gunicorn
Postgres
```

```
Redis
Celery
```

🎯 Final Architecture

```
            User
             │
             ▼
           Nginx
             │
      ┌──────┴──────┐
      ▼             ▼
   React        Django
                    │
            ┌───────┴───────┐
            ▼               ▼
       PostgreSQL       Redis
                          │
                          ▼
                        Celery
                          │
                          ▼
                       AI Model
```

👩‍💻 Final Result

এই project শেষ করলে তুমি পারবে:

- Production Django backend
- Real AI system integration
- Fullstack React + Django
- Docker deployment
- Redis + Celery async processing
- Nginx production server

এটা **portfolio level project** হবে।

তুমি যেহেতু সত্যিই **production-level Django stack শিখতে চাও**, আমি তোমাকে একটি **clear step-by-step build plan + folder structure** দিচ্ছি।

এটা follow করলে তুমি ধাপে ধাপে **AI Surveillance System (React + Django + Redis + Celery + Docker + Nginx)** বানাতে পারবে। 🚀

🧱 1️⃣ Complete Project Folder Structure

```
ai-surveillance-system
│
├── backend
│   ├── config
│   │   ├── __init__.py
│   │   ├── settings.py
│   │   ├── urls.py
│   │   ├── asgi.py
│   │   └── wsgi.py
│   │
│   ├── cameras
│   │   ├── models.py
│   │   ├── views.py
│   │   ├── serializers.py
│   │   ├── urls.py
│   │
│   ├── alerts
│   │   ├── models.py
│   │   ├── views.py
│   │   ├── serializers.py
│   │
│   ├── detection
│   │   ├── ai_model.py
│   │   ├── tasks.py
│   │
│   ├── celery.py
│   ├── manage.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend
│   ├── src
│   │   ├── pages
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Cameras.jsx
│   │   │   └── Alerts.jsx
│   │   └── components
│   │
│   ├── package.json
│   └── Dockerfile
│
```

```
├── nginx
│   └── default.conf
│
├── docker-compose.yml
└── .env
```

⚙️ 2️⃣ Backend Setup (Django)

Create project

```
django-admin startproject config backend
cd backend
```

Create apps:

```
python manage.py startapp cameras
python manage.py startapp alerts
python manage.py startapp detection
```

🗄️ 3️⃣ Database Models

Camera model

```
class Camera(models.Model):
    name = models.CharField(max_length=100)
    location = models.CharField(max_length=200)
```

Alert model

```
class Alert(models.Model):
    camera = models.ForeignKey(Camera, on_delete=models.CASCADE)
    timestamp = models.DateTimeField(auto_now_add=True)
    confidence = models.FloatField()
```

🌐 4️⃣ Django REST API

Install DRF

```
pip install djangorestframework
```

Serializer example:

```
class CameraSerializer(serializers.ModelSerializer):
    class Meta:
        model = Camera
        fields = "__all__"
```

View example:

```
class CameraViewSet(ModelViewSet):
    queryset = Camera.objects.all()
    serializer_class = CameraSerializer
```

🧠 5️⃣ AI Model Integration

তোমার CNN-LSTM model এখানে integrate হবে।

Example:

```
backend/detection/ai_model.py
```

```
def predict(frame):
    result = model.predict(frame)
    return result
```

⚡ 6️⃣ Redis + Celery

Install:

```
pip install celery redis
```

Celery config:

```
backend/celery.py
```

```python
from celery import Celery

app = Celery("backend")
app.config_from_object("django.conf:settings", namespace="CELERY")
```

Example task:

```
tasks.py
```

```python
@shared_task
def process_video(video_id):
    run_ai_model(video_id)
```

🐳 7️⃣ Docker Setup

docker-compose.yml

```yaml
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"

  postgres:
    image: postgres:15

  redis:
    image: redis:7

  celery:
    build: ./backend
    command: celery -A config worker -l info

  nginx:
    image: nginx
```

🌐 8️⃣ Nginx Config

```
nginx/default.conf
```

```
server {
    listen 80;

    location / {
        proxy_pass http://frontend:5173;
    }

    location /api/ {
        proxy_pass http://backend:8000;
    }
}
```

📊 9️⃣ React Dashboard

Pages:

```
Dashboard
Cameras
Alerts
```

Example API call:

```
fetch("http://localhost/api/cameras")
```

🚀 🔟 Run the System

Start everything:

```
docker compose up --build
```

Services run করবে:

```
React
Django
Postgres
Redis
```

```
Celery
Nginx
```

🧠 Final Production Architecture

```
User
 │
 ▼
Nginx
 │
 ├── React
 └── Django API
        │
        ▼
     PostgreSQL
        │
        ▼
       Redis
        │
        ▼
      Celery
        │
        ▼
      AI Model
```

🎯 Result

এই project শেষ করলে তুমি শিখবে:

- Django REST API
- PostgreSQL database
- Redis caching
- Celery background jobs
- Docker containerization
- Nginx reverse proxy
- Production deployment

এটা **portfolio-level production project** হবে।

🚀 10 Day Production Django + React Plan

📅 Day 1 — Backend Foundation (Django + DRF)

Goal: Basic API ready

Tasks:

- Django project create
- DRF install
- Apps create
  - cameras
  - alerts
- Models

Example:

```
Camera
Alert
```

Run API:

```
/api/cameras
/api/alerts
```

Learn:

- DRF ViewSet
- Serializer
- Router

📅 Day 2 — PostgreSQL

Goal: SQLite → Postgres

Install:

```
psycopg2
python-decouple
```

settings.py:

```
DATABASES = {
 'default': {
  'ENGINE': 'django.db.backends.postgresql',
```

```
    'NAME': config('DB_NAME'),
    'USER': config('DB_USER'),
    'PASSWORD': config('DB_PASSWORD'),
    'HOST': 'postgres',
    'PORT': '5432',
  }
}
```

docker-compose add:

```
postgres
```

Run:

```
python manage.py migrate
```

Learn:

- production database
- docker database connection

📅 Day 3 — Redis

Goal: Redis understand + connect

Add service:

```
redis
```

Install:

```
pip install redis django-redis
```

Use for:

```
cache
rate limit
queue
```

settings:

```
CACHES = {
 "default": {
  "BACKEND": "django_redis.cache.RedisCache",
  "LOCATION": "redis://redis:6379/1"
 }
}
```

📅 Day 4 — Celery

Goal: Background worker

Install:

```
pip install celery
```

Create:

```
celery.py
tasks.py
```

Example:

```
@shared_task
def detect_video(video_id):
    run_ai_model(video_id)
```

Run worker:

```
celery -A config worker -l info
```

📅 Day 5 — AI Model Integration

You already trained:

```
CNN + LSTM suspicious detection
```

Create:

```
detection/ai_model.py
```

Example:

```
def predict_clip(frames):
    result = model.predict(frames)
    return result
```

Celery task call:

```
video upload
→ celery task
→ model predict
→ save alert
```

📅 Day 6 — React Dashboard

Create pages:

```
Dashboard
Cameras
Alerts
```

Example:

```
fetch("/api/cameras")
```

Add:

```
camera list
alerts
live stream
```

📅 Day 7 — Docker Full Stack

Services:

```
backend
frontend
postgres
redis
celery
```

Run:

```
docker compose up --build
```

Learn:

```
volume
network
container communication
```

📅 Day 8 — Gunicorn

Replace Django dev server.

Install:

```
pip install gunicorn
```

Run:

```
gunicorn config.wsgi:application
```

Docker CMD:

```
gunicorn config.wsgi:application --bind 0.0.0.0:8000
```

📅 Day 9 — Nginx

Add reverse proxy.

Flow:

```
User
 ↓
Nginx
 ↓
React + Django
```

Config:

```
/api → django
/ → react
```

📅 Day 10 — Final Production System

Final architecture:

```
User
 │
 ▼
Nginx
 │
 ├── React Frontend
 │
 └── Django API
        │
        ▼
    PostgreSQL
        │
        ▼
      Redis
        │
        ▼
      Celery Worker
        │
        ▼
       AI Model
```

Final features:

```
✔ camera API
✔ suspicious detection
✔ alerts
✔ dashboard
✔ async processing
✔ docker deployment
```

⚠️ Important Advice (10 day success)

Daily study time:

```
6–8 hours
```

Focus:

```
Build first
Understand later
```

💡 Important for YOUR Project

তুমি যেহেতু already বানাচ্ছো:

```
AI CCTV Suspicious Detection
```

এই stack perfect হবে:

```
React dashboard
Django API
CNN-LSTM detection
Celery background processing
Redis queue
Postgres database
Docker deployment
Nginx reverse proxy
```

এটা **very strong portfolio project** হবে।

✅ যদি তুমি চাও, আমি তোমাকে **exact full docker-compose production setup** দিতে পারি যেখানে থাকবে:

```
React
Django
Postgres
Redis
Celery
Gunicorn
Nginx
```

এটা **real production architecture** হবে (very important skill)।